**PES UNIVERSITY**

Department of Computer Science and Engineering

**DATABASE MANAGEMENT SYSTEM**

Experiential Learning Level 3 – Mini Project Report

# MULTI-CARRIER SHIPMENT TRACKING AND DELAY ANALYTICS SYSTEM

**Project Name: ShipTrack**


| Student Name | SRN |
|---|---|
| Purushothama K C | PES1UG25AM810 |
| Rottela Haritej | PES1UG24AM231 |


August – December 2026

# 1. Project Title

**Multi-Carrier Shipment Tracking and Delay Analytics System (ShipTrack).**

ShipTrack is a web-based logistics management application designed to manage customer orders, shipments, multiple carriers, warehouses, shipment progress, tracking events, and delivery-delay analytics through a centralized platform.

# 2. Student SRNs and Names

| Sl. No. | Student Name | SRN |
|---|---|---|
| 1 | Purushothama K C | PES1UG25AM810 |
| 2 | Rottela Haritej | PES1UG24AM231 |

# 3. Abstract

The Multi-Carrier Shipment Tracking and Delay Analytics System provides a centralized platform for monitoring shipments handled by multiple logistics carriers. The application includes order management, shipment management, carrier and warehouse information, shipment tracking, analytics, and reports. Users can create and manage orders, associate shipments with carriers, view shipment statuses, and inspect tracking information.

The project follows a hybrid database approach. Structured business records are handled by the SQL database layer, while tracking-event documents are stored in MongoDB. The tracking-event structure includes shipment ID, tracking number, event type, location, GPS coordinates, timestamp, carrier, vehicle information, description, and source device. Next.js API routes connect the interface with the database layer.

The objective is to demonstrate how relational business records and document-oriented tracking data can work together in a logistics application and support shipment visibility and operational reporting.

# 4. Technology Stack

| Layer | Technology / Purpose |
|---|---|
| Frontend | Next.js 14, React 18, TypeScript, Tailwind CSS |
| Backend / API | Next.js App Router API routes |
| SQL layer | Prisma and SQL scripts; confirm active provider in prisma/schema.prisma and environment configuration |
| NoSQL database | MongoDB; database: shiptrack_mongodb; collection: tracking_events |
| Maps | Leaflet, React-Leaflet, OpenStreetMap |
| Charts | Recharts |
| Validation and forms | Zod, React Hook Form |
| Authentication utilities | bcryptjs, jsonwebtoken |
| Development tools | Visual Studio Code, Node.js, npm, Git, GitHub |

# 5. SQL – Database Commands

The repository organizes SQL work under **database/queries/**:

- **01_schema_ddl.sql** – schema and table definitions (DDL).
- **02_triggers_and_functions.sql** – trigger and function definitions.
- **03_views.sql** – database views.

- **04_indexes.sql** – index creation.
- **05_sample_dml_queries.sql** – sample DML and retrieval queries.

### 5.1 DDL example

```
-- Illustrative example; verify against the actual schema
CREATE TABLE orders (
  id VARCHAR(64) PRIMARY KEY,
  order_number VARCHAR(40) UNIQUE NOT NULL,
  customer_id VARCHAR(64) NOT NULL,
  customer_name VARCHAR(120) NOT NULL,
  order_date TIMESTAMP NOT NULL,
  total_amount DECIMAL(12,2) NOT NULL,
  status VARCHAR(20) NOT NULL,
  items_count INTEGER NOT NULL
);
```

### 5.2 DML examples

```
-- Example insert; verify actual table and column names
INSERT INTO orders
(order_number, customer_id, customer_name, order_date,
 total_amount, status, items_count)
VALUES ('ORD-DEMO-001','CUST-DEMO-001','Demo Customer',
 CURRENT_TIMESTAMP,999.00,'PENDING',1);

SELECT * FROM orders
WHERE order_number = 'ORD-DEMO-001';

UPDATE orders SET status='PROCESSING'
WHERE order_number='ORD-DEMO-001';

DELETE FROM orders
WHERE order_number='ORD-DEMO-001';
```

### 5.3 JOIN and aggregation example

```
-- Query pattern; verify table and key names in the actual schema
SELECT status, COUNT(*) AS order_count
FROM orders
GROUP BY status
HAVING COUNT(*) > 0;
```

For evaluation, demonstrate the actual SQL scripts and explain the DDL constraints, DML operations, joins and aggregations, triggers, functions, views, and indexes that are implemented. The examples above are illustrative and must be checked against the final schema.

## 6. NoSQL / Vector Database Commands

The project uses **MongoDB as its NoSQL database**; it does not use a vector database. The source file **src/lib/mongodb.ts** specifies the database name as **shiptrack_mongodb** and the collection name as **tracking_events**.

The tracking-event document structure represented in the project code includes shipment_id, tracking_no, event_type, location, latitude, longitude, timestamp, carrier, vehicle_info, description, and source_device.

### 6.1 MongoDB CRUD examples

```
use shiptrack_mongodb

// Read events for a tracking number
db.tracking_events.find({ tracking_no: "TRK-NOV-03" })

// Insert a sample event
db.tracking_events.insertOne({
  shipment_id: "SHIP-DEMO-001",
  tracking_no: "TRK-DEMO-001",
  event_type: "PICKED_UP",
  location: "Bangalore Warehouse",
  latitude: 12.9716,
  longitude: 77.5946,
  timestamp: new Date().toISOString(),
  carrier: "BlueDart",
  description: "Shipment picked up"
})
```

```
// Update an event
db.tracking_events.updateOne(
  { tracking_no: "TRK-DEMO-001" },
  { $set: { description: "Shipment dispatched" } }
)

// Delete the sample event
db.tracking_events.deleteOne({ tracking_no: "TRK-DEMO-001" })
```

These are demonstration commands. Verify the application's actual MongoDB document structure and methods in src/lib/mongodb.ts before running commands against project data.

## 7. Next.js – Project Tree Structure

```
ShipTrack/
■■■ database/
■   ■■■ queries/
■       ■■■ 01_schema_ddl.sql
■       ■■■ 02_triggers_and_functions.sql
■       ■■■ 03_views.sql
■       ■■■ 04_indexes.sql
■       ■■■ 05_sample_dml_queries.sql
■■■ prisma/
■   ■■■ schema.prisma
■■■ scripts/
■   ■■■ seed.js
■■■ src/
■   ■■■ app/
■   ■   ■■■ api/
■   ■   ■   ■■■ analytics/
■   ■   ■   ■■■ auth/
■   ■   ■   ■■■ carriers/
■   ■   ■   ■■■ orders/
■   ■   ■   ■■■ reports/
■   ■   ■   ■■■ shipments/
■   ■   ■   ■■■ tracking/
■   ■   ■   ■■■ users/
■   ■   ■   ■■■ warehouses/
■   ■   ■■■ analytics/
■   ■   ■■■ carriers/
■   ■   ■■■ dashboard/
■   ■   ■■■ login/
■   ■   ■■■ orders/
■   ■   ■■■ reports/
■   ■   ■■■ shipments/
■   ■   ■■■ tracking/
■   ■   ■■■ users/
■   ■   ■■■ warehouses/
■   ■■■ components/
■   ■■■ lib/
■       ■■■ auth.ts
■       ■■■ db.ts
■       ■■■ mongodb.ts
■       ■■■ cities.ts
■       ■■■ seed-data.ts
■■■ .env.example
■■■ .gitignore
■■■ README.md
■■■ package.json
■■■ package-lock.json
```

Frontend pages are located under src/app. Backend endpoints are Next.js API route handlers under src/app/api. Shared database and authentication utilities are located in src/lib.

## 8. Sample Screenshots of Application Code and Demo

Insert genuine screenshots captured from the running application and source code in this section. Recommended evidence includes:

- Dashboard – shipment summary cards, charts, recent shipments, delayed shipments, and live map.
- Orders Management – create-order form and saved orders list.
- Shipments Management – tracking number, order, carrier, customer, current location, and status.
- Tracking Page – shipment timeline and raw MongoDB events log.

- Live Shipments Map – map visualization and shipment locations.
- Analytics and Reports – generated reports and carrier or shipment performance.
- SQL Implementation – schema, triggers/functions, views, indexes, and DML scripts.
- MongoDB Implementation – src/lib/mongodb.ts and MongoDB Compass or mongosh showing tracking_events documents.
- API Implementation – order and tracking API route examples.

**Screenshot placeholders:** Add actual screenshots below these descriptions before final submission. Do not use mock screenshots as proof of implemented features.

## 9. Evaluation Rubrics – Completion Status

The maximum marks below follow the PES University evaluation sheet supplied by the team. The status column is left for verification against the final running implementation and faculty demonstration.

| Evaluation Component | Max. | Evidence to demonstrate | Status |
|---|---|---|---|
| ER diagram | 2 | Attach ER diagram; explain entities, relationships, keys, and constraints. | To be verified |
| Relational schema with keys, relationships, constraints | 3 | Show final relational schema. | To be verified |
| SQL database implementation – DDL with constraints | 4 | Demonstrate SQL schema scripts. | To be verified |
| NoSQL/Vector database creation | 4 | Show MongoDB database and tracking_events collection. | To be verified |
| Technology justification | 1 | Explain SQL for structured data and MongoDB for tracking events. | To be verified |
| DML operations | 4 | Demonstrate insert, read, update, and delete. | To be verified |
| Nested/correlated/recursive queries and JOIN queries | 4 | Show implemented SQL queries. | To be verified |
| Advanced SQL: procedure/function, trigger, CTE | 6 | Demonstrate concepts actually present in SQL scripts. | To be verified |
| NoSQL/Vector feature (RUD) | 7 | Demonstrate MongoDB read, update, delete, and insert. | To be verified |
| Next.js interface | 3 | Demonstrate application pages. | To be verified |
| Backend/API and database integration | 2 | Demonstrate API/database read and write. | To be verified |
| Authentication and minimum two roles / role-based access | 4 | Demonstrate login, roles, and access restrictions. | To be verified |
| Documentation, Git/GitHub | 2 | Show README, report, repository, and commit history. | To be verified |
| Originality, individual contribution, modification/viva | 4 | Explain each member's individual contribution. | To be verified |
| TOTAL | 50 | | |

Completion status and marks must be verified during the actual project demonstration. Do not mark a feature completed unless it has been implemented and tested.

# Appendix A – Official Project Report Requirements

The following image was supplied by the project team and is included as the reference for the nine required report sections.



You are required to submit your **DBMS Project Report** covering the following sections:

1. Project Title
2. Student SRNs and Names
3. Abstract
4. Technology Stack
5. SQL – Database Commands
6. NoSQL/Vector Database Commands
7. Next.js – Project Tree Structure
8. Sample Screenshots of Application Code and Demo
9. Evaluation Rubrics – Completion Status

# Final Verification Before Submission

- Confirm the active SQL provider and database connection using prisma/schema.prisma and local environment configuration. Never expose credentials.
- Check every SQL example against the actual SQL scripts before execution.
- Verify that shiptrack_mongodb and tracking_events contain the application's real tracking documents.
- Insert actual screenshots of the application, code, and database records in Section 8.
- Attach the final ER diagram and relational schema.
- Update the rubric status only after testing and demonstration.
- Never include .env, passwords, database credentials, or private tokens in the report or GitHub repository.